

MediQueue

Smart Clinic Queue Management System

Assessment Type:	Individual 48-Hour Software Engineering Examination
Live Web Application:	https://mediqueue-25vl.onrender.com
Hospital TV Display Screen:	https://mediqueue-25vl.onrender.com/display
In-App Documentation Hub:	https://mediqueue-25vl.onrender.com/docs
GitHub Source Repository:	https://github.com/mhiskall282/ug-swe-exams
Automated Test Suite:	43 PHPUnit Tests / 160 Assertions (100% Pass)
Technology Stack:	Laravel 12, PHP 8.2, Tailwind CSS v4, Managed PostgreSQL, Vite 7
Author / Candidate:	Lead Senior Software Engineering Candidate
Submission Date:	August 14, 2026

Status: Production Ready, Deployed on Render Cloud PaaS, Fully Tested and Verified.

1. Executive Summary & Problem Definition

MediQueue is a modern, accessible, enterprise-grade clinic queue management web application designed to eliminate physical waiting lines in outpatient healthcare facilities. In conventional outpatient environments, physical queues produce severe waiting room overcrowding, patient anxiety, opaque triage prioritization, and elevated nosocomial contagion risks. MediQueue replaces physical lines with a resilient, transactional digital queue engine.

The project was engineered, tested, and containerized within an intensive 48-hour examination scope, delivering complete role-isolated workflows for patients, clinical staff, administrators, and public waiting room screens.

43 Tests (100% Pass) Automated PHPUnit	160 Assertions Zero Test Failures	~90 UCP (~46h) Algorithmic Estimation	0 Collisions Pessimistic Concurrency
---	--------------------------------------	--	---

2. System Architecture & Concurrency Engineering

MediQueue adopts a **Modular Layered Monolith** architecture (ADR-002). Domain logic, ticket sequencing, and state transitions are centralized inside App\Services\QueueService. To guarantee race-condition immunity during concurrent patient arrivals, all queue assignments utilize pessimistic row-level locking (lockForUpdate) inside atomic transactions.

Tier / Layer	Technology & Implementation	Responsibility & Concurrency Guarantees
Presentation Layer	Laravel Blade + Tailwind CSS v4	Responsive glassmorphic UI, real-time polling updates, accessible forms
Security & Routing	RoleMiddleware & RateLimiter	Multi-role isolation (patient/staff/admin), brute-force route throttling
Business Logic	QueueService.php	Atomic ticket numbering (e.g. GC-005), state machine transitions, event dispatch
Data & Persistence	Eloquent ORM Models	User, Service, QueueEntry, Notification, Setting, AuditLog schemas
Database Store	PostgreSQL / SQLite	ACID transactional consistency with foreign keys, compound indexes, row locks

3. Queue Deterministic State Machine

Queue entries progress through a formal finite state machine. Invalid state mutations (such as attempting to complete an uncalled ticket) throw validation exceptions.

Initial State	Target State	Actor	Transition Trigger & Business Logic
WAITING	CALLED	Staff	Staff calls next ticket by priority FIFO; sets called_at and broadcasts alert.
WAITING	CANCELLED	Patient / Admin	Patient cancels ticket before being called (Terminal state).
CALLED	IN_SERVICE	Staff	Patient reports to room; consultation starts and timers begin.
CALLED	SKIPPED	Staff	No-show patient moved to skipped pool after callout timeout.
SKIPPED	CALLED	Staff	Skipped patient reports to desk; recalled to active consultation call.

IN_SERVICE	COMPLETED	Staff	Consultation concluded; computes duration and archives ticket (Terminal state).
------------	-----------	-------	---

4. Requirements Traceability Matrix (RTM)

All requirements specified in the SRS are fully mapped to concrete implementing classes and automated verification tests:

Req ID	Requirement Description	Priority	Implementation Class	Verification Test
REQ-AUTH-001	Patient Registration & Hashing	MUST	RegisterController	AuthTest::test_patient_can_register
REQ-AUTH-006	Role-Based Access Control (RBAC)	MUST	RoleMiddleware	AuthorizationTest (8 cases)
REQ-QUEUE-001	Atomic Ticket Numbering	MUST	QueueService::join	QueueLifecycleTest::test_sequential
REQ-QUEUE-004	Duplicate Active Ticket Lock	MUST	QueueService::join	QueueLifecycleTest::test_duplicate
REQ-QUEUE-008	Staff Call Next Operation	MUST	StaffQueueController	QueueLifecycleTest::test_call_next
REQ-DISP-001	Hospital Public TV Departure Screen	SHOULD	DisplayController	AdminEnhancementsTest::test_display
REQ-NOTIF-001	Transactional Email & In-App Alerts	SHOULD	QueueNotificationMail	AdminEnhancementsTest::test_email
REQ-SETT-001	Clinic Settings & Password Reset	SHOULD	SettingController / UserController	AdminEnhancementsTest::test_reset
REQ-REP-001	Analytics, CSV Export & Investigation	SHOULD	ReportController	AdminEnhancementsTest & Manual Tests
REQ-AUDIT-001	Immutable Security Audit Ledger	MUST	AuditLog Model	SmokeTest & LifecycleTests

5. Software Effort Estimation (Use Case Points)

Using Gustav Karner's algorithmic Use Case Points formula, unadjusted UCP was computed as 164. Technical Complexity Factor (TCF = 0.935) and Environmental Complexity Factor (ECF = 0.590) adjusted total effort to **~90 UCP (46 person-hours)**, fitting the 48-hour examination scope.

6. Reporting, CSV Export & Forensic Investigation

To satisfy hospital governance and clinical inquiry requirements, MediQueue provides an advanced **Reporting & Analytics Hub** (/admin/reports) alongside a **Forensic Chain of Custody Tracker** (/admin/reports/investigate/{id}):

- **Custom Date & Staff Filtering:** Filter attendance by department, date range, attending doctor, and status.
- **Real-Time CSV Export:** Download a streaming CSV ledger of all queue entries including wait duration and consultation minutes.
- **Executive Email Dispatch:** Instant one-click dispatch of operational summaries to the clinic administrator.
- **Forensic Chain of Custody:** Full accountability tracking showing who registered the patient, which staff member called them, when service started and concluded, and all associated immutable audit log entries with client IP addresses.

7. Quality Assurance & Automated Test Results

All 43 automated PHPUnit tests (160 assertions) executed with 100% pass rate:

Test Suite File	Scope Verified	Assertions	Result
AuthTest.php	Registration, password hashing, login, logout, sessions	16	100% PASS
AuthorizationTest.php	RoleMiddleware, vertical/horizontal privilege isolation	28	100% PASS
QueueLifecycleTest.php	Atomic numbers, lockForUpdate, call/start/complete/skip	52	100% PASS
AdminEnhancementsTest.php	Hospital TV screen, settings, password reset, email dispatch	34	100% PASS
SmokeTest.php	Guest, patient, staff, admin, /docs 200 HTTP rendering	30	100% PASS

8. Seeded Demo Accounts for Evaluation

The following credentials can be used immediately to evaluate the live deployment:

Role	Email Address	Password	Accessible Surfaces & Capabilities
Administrator	admin@mediqueue.test	password	Full system control, services, password resets, settings, reports, audit.
Doctor / Staff	dr.sarah@mediqueue.test	password	Clinical queue console, Call Next CTA, start/complete, skip, recall.
Nurse / Staff	nurse.james@mediqueue.test	password	Nursing department queue triage and management.
Patient	john.doe@example.com	password	Queue ticket issuance, real-time live position tracking, history.

9. Conclusion & Submission Links

MediQueue demonstrates full compliance with all advanced software engineering capstone examination standards. The application is completely functional, architecturally disciplined, concurrency-safe, responsive, secure, and live in production.

Resource	Production URL / Destination
Live Web Application	https://mediqueue-25vl.onrender.com
Hospital TV Departure Screen	https://mediqueue-25vl.onrender.com/display
In-App Documentation Hub	https://mediqueue-25vl.onrender.com/docs
Reporting & Analytics Portal	https://mediqueue-25vl.onrender.com/admin/reports
GitHub Source Repository	https://github.com/mhiskall282/ug-swe-exams